

HURRICANESOFT 颶風軟體

TrustForge Hermes

Evidence-native Decision Intelligence Agent

商業化產品提案報告 | 加密市場資訊的可治理信任層

文件資訊	內容	文件資訊	內容
產品團隊	中再參與 HurricaneSoft	提案角色	商業化產品與技術方案
命題場域	智慧交易 / 金融資訊	版本	2026-08-01
文件用途	完整書面提案；非 6 分鐘上台講稿	讀者	金融企業、產品、工程與治理團隊

核心主張 TrustForge 不把 AI 當成會回答問題的黑箱，而是把多源資料、確定性信任推理、模型協作、治理與稽核串成可部署、可觀測、可回復的 AI Agent 產品。

摘要與產品定位

TrustForge Hermes 是面向金融資訊與高風險研究場景的 evidence-native decision intelligence agent。產品將行情、新聞、社群、監管、鏈上與企業資料導入同一個 evidence contract，先將內容拆解為可驗證 Claims，再透過 deterministic trust reasoning 計算來源信譽、獨立佐證、時效衰減、重複與異常訊號，最後才讓 Amazon Bedrock 將受控的 TrustedBrief 組織為可讀報告。

產品交付的不是單一分數或一句預測，而是可回溯的 decision record：結論、信心、限制、反方證據、來源、時間戳、模型版本、執行紀錄與產物雜湊都能被重播與抽查。這使 TrustForge 能從競賽原型升級為可嵌入研究、風控、客服、合規與營運流程的 Trust Layer。

產品邊界 TrustForge 衡量資訊的可信度、完整度與可稽核性，不把輸出包裝成投資建議，也不把 Trust Score 宣稱為漲跌勝率。

商業化價值

- 降低研究人員逐條核對來源與重複轉載的時間成本。
- 讓風控與合規團隊能抽查 claim-to-source、模型版本與批准紀錄。
- 以 API、artifact 與 telemetry 接入既有交易、客服、研究與稽核系統。
- 以可替換 provider、memory、policy、calibrator 與 backend 降低單一模型或單一雲服務的鎖定風險。

一、問題定義與企業場景

加密市場的資料量不是唯一問題，真正的風險是資料的權重、時效與關聯不透明。鏈上事實、價格資料、監管公告、新聞敘事與匿名社群不應被視為同等證據；同一篇內容被多站轉載也不能被誤算成多個獨立來源。傳統 RAG 或摘要器可以快速生成流暢文字，卻不一定能回答「這個主張來自哪裡、何時取得、是否被獨立來源支持、什麼情況會推翻它」。

企業問題	產品回應	可驗證產物
研究效率	以 ingestion 、 claim extraction 、 deduplication 與 evidence grouping 整理多源訊號	Analysis Journey 、 Final Report
風險溝通	並列支持與反方證據，揭露 uncertainty 、 limits 與 could-flip 條件	Evidence List 、 Trust breakdown
治理與稽核	以 run_id 、 Execution Log 、 model version 與 manifest 串接決策	Execution Log 、 SHA-256 manifest
持續改善	以 learning event 與延遲 outcome 建立校準候選，通過 gate 後才進入審核	Training contract 、 candidate 、 approval receipt

二、產品總體架構

TrustForge 由資料平面、推理平面、控制平面與產品介面組成。每個平面都有明確責任與可替換邊界，避免將所有決策責任集中在單一 LLM prompt。

2.1 三層執行管線

層級	責任	核心模組
Ingestion Layer	接收、清理、標準化與標記外部資料	prices 、 HOYA BIT 、 news 、 social 、 regulatory 、 on-chain 、 whale trades 、

		CoinGecko 、 CMC 、 DefiLlama 、 Etherscan
Reasoning Layer	主張抽取、立場分類、去重、交叉佐證、信任分數與 uncertainty	pipeline、Trust Kernel、corroboration、evidence grouper、freshness、direction resolution
Agent / Delivery Layer	受界限的工具調度、報告組裝、產物交付與 API 回應	Hermes orchestrator、Bedrock、report delivery、Final Report、Evidence List、Execution Log、Snapshot

2.2 前後端產品面

Frontend 以 React/Vite 與 Hermes Workspace 呈現產品能力；Backend 以 Python API、analysis flow 與 formal run runtime 執行任務。兩者透過結構化 API 與同一個 run_id 連接。

產品面	主要能力
Research Workspace	Analyze、Compare、History、Source Status、Whale Activity、Evidence Trail、信心與證據分解、facts/inferences ladder、反方證據與 report download
Operations Console	Training Status、Upgrade Queue、Module Telemetry、Budget、Carbon、Health、成本與執行狀態
Governance Console	Admin、AGOS、policy、memory、prompt version、approval、rollback、audit record

Integration Surface	API response 、 snapshot 、 artifact 、 manifest 、 Final Report 、 Evidence List 、 Execution Log 、 Source/Config
---------------------	--

2.3 AWS 目前部署拓樸

現行公開拓樸以 EC2 t3.micro + nginx 為基礎：nginx 對外提供 HTTPS 與 frontend，Python API 收斂在 127.0.0.1:8080；Amazon Bedrock 是唯一 LLM 入口，S3 保存資料與產物，IAM Instance Role、SSM Session Manager 與 CloudWatch 支援最小權限與運維觀測。App Runner 是未採用的舊構想，不列為目前已部署能力。

元件	目前責任	治理重點
EC2 + nginx	承載 React/Vite frontend 與 Python API reverse proxy	網路邊界、TLS、127.0.0.1 API binding
Amazon Bedrock	claim extraction、stance classification、report assembly 等語言任務	只做語言工作，不直接決定 Trust Score
S3	訓練資料、模型 artifact、部署與 manifest 保存	IAM、雜湊、保留與版本化
IAM / SSM / CloudWatch	無長期 SSH key 的維運、權限與指標	最小權限、審計、健康檢查

三、核心功能與獨立系統

3.1 Data Ingestion 與資料契約

Ingestion 將外部資料轉成統一的 `source`、`fetch_at`、`content_reference`、`asset`、`data_mode` 與 `freshness` 欄位，並保留失敗原因與覆蓋缺口。連接器各自獨立，單一來源故障不應讓整體分析假裝完整。

- 行情與企業資料：HOYA BIT 及價格 OHLCV，作為方向基準與 `delayed outcome ground truth`。
- 新聞與社群：RSS / API、X / Reddit 等，用於事件與情緒訊號，但不與官方資料等權。
- 鏈上與巨鯨：Etherscan、Whale Alert、Arkham、CoinMarketCap 等，辨識大額轉帳、錢包歸因與流動性背景。
- 監管與官方公告：保留正式發布時間與原文範圍，支援高風險事件追溯。
- 資料品質：`cache`、`safe fetch`、`connector reliability`、`freshness`、`data integrity` 與 `SHA-256 manifest`。

3.2 Trust Kernel 與信任推理

Trust Kernel 是產品的 `deterministic trust reasoning` 核心。它在 LLM 行文之前處理可重現的規則與計算：來源先驗、交叉佐證、時效衰減、重複計數、矛盾、操縱風險、`confidence` 與 `uncertainty`。相同輸入、規則與版本應得到相同結果，便於測試、回放與稽核。

重要邊界 Trust Kernel 的分數是資訊信任與完整度訊號，不是交易勝率；Bedrock 不被授權單獨產生市場方向。

3.3 Formal Run、計分與交付

- `formal_run_coordinator`、`budget reservation`、`idempotency` 與 `lease` 防止重複執行。
- `analysis flow` 將 `claim extraction`、`stance`、`deduplication`、`corroboration`、Trust Kernel 與 `uncertainty` 串成可觀測步驟。

- 每次 run 以 run_id 串接 Final Report、Evidence List、Execution Log、Snapshot、Source/Config 與 API response。
- 遇到 timeout、資料不足、權限錯誤或模型異常，系統進入 degraded、abstain 或 fail-closed，不硬湊答案。

3.4 Agent Runtime、Memory 與 Policy

Hermes orchestrator、AgentCore adapter/runtime、tool registry、memory OS 與 policy compiler/executor 共同形成受界限的 Agent Runtime。Memory plane 保存 run、artifact、evidence、learning event 與 replay history；policy plane 記錄 source、analysis、report、evaluation 與 improvement 的實際消費者與允許行為。

AgentCore 在產品中是可替換的執行整合層，而不是把所有 TrustForge domain logic 外包給第三方。工具呼叫、預算、權限、錯誤與狀態仍由 TrustForge 的 contract、guards 與 audit 控制。

3.5 Upgrade Control 與自動升級治理

Improvement、upgrade control、canary、activation lock、deployment evidence 與 rollback record 形成升級控制平面。自主循環可以 refresh_sources、measure_quality、replay_history、diagnose_improvement 並提出 candidate，但 candidate 必須通過 benchmark、安全、成本、相容性與人工批准，才可進入 activation。

狀態	系統行為	不可省略的證據
Candidate	建立新模型、規則或模組候選，不影響 production	metrics、artifact hash、training contract
Review	進行 quality、security、cost、compatibility 與 adversarial review	reviewer attestation、decision record

Activate	經人工批准後才切換，保留 canary 與 activation lock	approval 、 deployment evidence 、 receipt
Rollback	發現異常時回到已知穩定版本	rollback decision record 、 health check 、 audit trail

3.6 可觀測性、成本與碳管理

- Module Telemetry：記錄模組執行次數、耗時、成功率、錯誤與版本。
- Budget Guard：限制 token、Bedrock 呼叫、Agent 次數與每日 USD 使用量，超限時停止或降級。
- Carbon：記錄 input/output token 與推理消耗，透過 Trust Kernel 本地計算、快取與必要時才呼叫 LLM，降低不必要算力。
- Audit：以 hermes audit、execution events、artifact registry 與 authenticated ledger 支援追責與重播。

四、模型策略：Bedrock、校準模型與訓練閉環

4.1 為什麼不只使用現成生成模型

Bedrock foundation model 擅長語言理解、資訊抽取與報告組裝，但它不應直接承擔企業對信任分數、校準、資料權重與升級批准的責任。生成模型的輸出具有機率性，且會受到 prompt、上下文與資料品質影響；金融資訊產品需要的是在固定資料、規則與版本下可重現、可比較、可稽核的信任訊號。

- 以本地 deterministic feature 與校準器保留可測試性，避免把治理責任藏在 prompt。
- 以自身資料與 delayed outcome 了解本產品的 confidence 是否過度自信，而不是只看通用模型 benchmark。
- 以 ModelHub governance 與 SageMaker execution backend 保持模型版本、artifact、指標與批准流程。

- 生成式 foundation model 仍只使用 Amazon Bedrock；自行訓練的是 TrustForge 的 task-specific calibrator，不是繞過競賽規則另訓第三方 foundation model。

4.2 Learning Event 與延遲標籤

每次資料抓取、分析、評分與報告完成後，系統會沉澱 learning event：包含來源、Claims、Trust Score、raw confidence、模型版本、run_id、Execution Log 與後續可對照的決策狀態。這是建立訓練資料的第一步，不代表當下立即重訓。

市場結果必須等待 outcome 成熟：T+1、T+7、T+14 分別代表分析時間點後第 1、7、14 天的實際結果。延遲標籤可以用來檢查信心是否合理、是否過度自信與校準是否需要修正，並避免把未來資訊洩漏回訓練資料。

4.3 訓練流程與 backend 邊界

1. Run 完成後寫入 learning event 與 immutable evidence / execution metadata。
2. T+1/T+7/T+14 outcome label 成熟後，建立 training row，執行資料契約、版本與 label leakage 檢查。
3. 以 Isotonic Regression 等校準方法產生 candidate，評估 ECE、Brier、holdout 與資料覆蓋範圍。
4. 由 SageMaker backend 執行 AWS 訓練任務，S3 保存輸入與 artifact；ModelHub 登記版本、指標、來源與狀態。
5. candidate 進入 review、canary、人工批准與 activation；不合格則退回或淘汰，production 版本不自動被取代。

實作現況 目前 training trigger 是顯式 CLI / 受控流程；程式與報告尚未宣稱存在無人值守 scheduler、每次 run 立即自動重訓或自動 activation。自動排程是後續 enhancement，人工否決權永久保留。

4.4 ModelHub 與 SageMaker 的責任分工

元件	角色	保存 / 執行內容
ModelHub	Governance surface	model version、training request、metrics、artifact provenance、candidate status、approval lifecycle
SageMaker AI	Training execution backend	從 S3 讀取資料、建立 Training Job、產生模型 artifact、回傳訓練指標
S3	Artifact storage	training dataset、calibrator artifact、manifest、版本與完整性雜湊

五、資料與證據治理

5.1 六類資料來源與用途

資料類型	範例來源	信號用途與治理
價格 / OHLCV	HOYA BIT、prices、CoinGecko	趨勢基準、T+ outcome、時間與覆蓋範圍
新聞	RSS / API、news connector	事件主張、發布時間、原文回溯
社群	X / Reddit、social connector	情緒與異常訊號；權重低於官方資料

鏈上	Etherscan 、 Solscan 、 on-chain	交易明細、合約互動、巨鯨動態
監管	官 方 公 告 、 Taiwan regulatory	政策與合規風險，保留正式來源
市場基本面	CMC 、 CoinGecko 、 DefiLlama	市值、流動性、排名與協議背景

5.2 Evidence Contract

每個 Claim 都應能連回 source、fetched_at、content_reference、asset、data_mode 與 related_claim。

Evidence List 不只是網址清單，而是把結論與原始證據、時間、來源家族、相互關係與限制綁定在一起。對矛盾訊號，系統保留支持與反方兩邊，不用簡單多數決掩蓋少數證據。

5.3 四項可交付物與企業交付

交付物	內容	企業用途
Final Report	結論、Trust Score、confidence、limits、could-flip、反方證據	研究決策與管理層閱讀
Evidence List	claim_id、來源、網址、時間、引用片段與關聯	風控、客服與合規抽查
Execution Log	工具、步驟、時間、成本、錯誤與狀態	重播、事故調查與責任追溯
Source / Config + Snapshot	程 式 、 配 置 、 資 料 / 模 型 版 本 、	可重現執行與部署驗證

	manifest 與快照	
--	--------------	--

六、安全、治理與營運控制

- 秘密管理：API key、AWS credentials、資料庫密碼不進 GitHub；使用環境變數、IAM Instance Role、SSM 與 secret boundary。
- 權限控制：tool registry 與 policy guards 限定工具、資料來源、模型與管理操作的 scope。
- Fail-closed：權限錯誤、資料不足、模型異常或預算超限時停止高風險輸出，改以 degraded 或 abstain 說明限制。
- 版本與稽核：每次分析與升級有 run_id、model version、artifact hash、approval receipt、deployment evidence 與 rollback record。
- 安全審查：升級 candidate 需通過 adversarial review、dependency / secret 檢查與 production gate；不以自動化速度取代人類批准。

治理定位 TrustForge 的設計精神與 ISO/IEC 42001 的風險、可追溯性、人工監督與持續監控方向一致；本提案不宣稱已取得 ISO 42001 認證或符合性證書。

七、商業化產品路線

階段	產品化目標	衡量方式
Phase 1：Research Trust Layer	把研究 / 市場資訊接入 Trust Layer API，提供 report、evidence、execution 與 snapshot	可溯源率、來源覆蓋、分析完成時間、人工核對節省
Phase 2：Governed	提供白標 API、比較快照、peer metrics、asset	API adoption、review

Decision Services	context 與審核 workflow	turnaround、錯誤與回滾時間
Phase 3 : Continuous Calibration	以成熟 outcome label 驅動校準 candidate、ModelHub / SageMaker 與 canary	ECE / Brier、candidate 通過率、人工批准與 rollback 指標
Phase 4 : Enterprise Control Plane	整合 IAM、audit、budget、carbon、telemetry 與多租戶 policy	成本可預測性、合規抽查成功率、服務可用性

產品護城河

- Evidence contract 與 claim-to-source lineage，而不是單一 prompt。
- 可測試的 Trust Kernel 與可替換模型 / 資料 / 儲存 backend。
- Bounded autonomous agent：能量測、診斷、提案，但不能越過人類批准。
- Calibration、replay、artifact provenance 與 rollback 形成持續改善閉環。
- 同一組產物同時服務前端使用者、工程團隊、金融風控與稽核人員。

八、驗收與限制

本產品以可驗證的工程契約作為驗收基礎，而非只以畫面是否能產生文字判斷成功。

驗收面	最低驗收條件
資料真實性	每個來源標示 data_mode、取得時間、覆蓋範圍；fixture、cache、live 不混淆

推理可重現	固定輸入、規則與版本可得到一致 Trust / Evidence 結果
交付完整	Final Report、Evidence List、Execution Log、Source/Config 與 manifest 可由同一 run_id 串接
訓練治理	training row 有成熟 outcome label ; candidate 有 metrics、artifact hash 與人工批准狀態
失敗安全	逾時、資料不足、權限錯誤與模型異常會 degraded、abstain 或 fail-closed

誠實限制

- TrustForge 不承諾市場方向預測或投資報酬。
- 部分 provider / AgentCore / DynamoDB backend 是可替換介面或支援路徑，不等於全部都是目前公開部署元件。
- 目前沒有宣稱每次 run 後立即自動訓練、無人值守 scheduler 或自動 activation。
- 操縱偵測與模型品質仍需以更多標籤、回放與實際營運資料持續驗證。

結語

TrustForge Hermes 的商業價值，不是再做一個會摘要市場新聞的聊天介面，而是把企業真正需要的信任、證據、成本、權限、版本與回復能力，做成可部署的 AI Agent control plane。

結論 從會回答的 AI，走向值得託付、能被抽查、能被回復的 AI Agent。

附錄：關鍵術語

術語	定義
Evidence-native	從資料契約與證據鏈開始設計，每個結論都有可追溯依據
Deterministic reasoning trust	固定輸入、規則與版本下可重現的信任推理
Training backend	實際讀取訓練資料、建立訓練任務與產出 artifact 的後端
Candidate	已產生但尚未經人工批准的模型、規則或升級候選版本
Audit trail	可追溯誰在何時以何種資料與版本做了什麼決策
Fail-closed	遇到不確定或錯誤時停止高風險輸出，不用猜測填補缺口